

# UpDown — Smart-Contract Audit Scope (CertiK)

Prepared for scoping a security audit prior to a public launch. It defines what is in scope, what is explicitly out of scope, and where the trust/centralization surface lives.

## TL;DR

- **Primary audit target (must):** the Solana Anchor program `parimutuel_pools` — the only production contract that custodies user funds. Program id `9H7k26HvHHnB4T6ErU7n2wVSFJhS1aigqFQGwvQyVuNG` . ~2,125 LOC Rust, 18 instructions.
- **Solana only.** No EVM migration is planned; the Arbitrum Solidity port has been archived to a side branch and is explicitly **out of scope**.
- **Not yet code (future audit):** UP token (SPL mint), Streamflow vesting/staking, Squads multisig, presale/claim contract, buyback. All documentation-only today.
- **Out of scope (do not pay to audit):** HyperLiquid, Pacifica, Privy custody, LI.FI bridge — third-party contracts. The Node/Next backend is a web pentest, not a contract audit.
- **Design review CertiK should include:** a single off-chain authority key resolves outcomes and drives all payouts, with no on-chain oracle. This is the core centralization risk and should be documented in the report.

## 1. In scope — PRIMARY: Solana `parimutuel_pools` (Anchor / Rust)

Path: `programs/parimutuel_pools/src/` ( `lib.rs` , `state.rs` , `errors.rs` , `events.rs` , `constants.rs` , `instructions/*.rs` , `tests/money_math.rs` ). Model: USDC (6-decimal SPL) escrowed in per-pool / per-tournament PDA vaults; a single `authority` (the backend scheduler) resolves and drives payouts. Supports 2-way (crypto up/down) and 3-way (sports home/draw/away) pools via `num_sides` .

### Instructions (18)

**Pool lifecycle** `initialize_pool` · `deposit` (only user-authored money-in) · `resolve` (crypto, price-based) · `resolve_with_winner` (sports/PM, explicit winner) · `claim` (payout + `fee_bps` ) · `refund` · `refund_bettor` (void / principal-only) · `close_losing_bet` · `sweep_vault_dust` · `close_pool` · `force_close_pool` .

**Tournament escrow** `initialize_tournament` · `register_participant` · `resolve_tournament` (authority sets winner + Completed) · `claim_tournament_prize` (prize – 5%) · `cancel_tournament` · `refund_participant` · `close_tournament` .

### State / PDAs

- Pool `["pool", pool_id]` , Vault `["vault", pool_id]` (token authority = pool PDA).
- UserBet `["bet", pool, user, side]` (per-side, allows hedging).
- Tournament `["tournament", id]` , Vault `["tournament_vault", id]` , Participant `["participant", tournament, user]` .
- Funds live only in the pool and tournament `vault` PDAs.

### Payout math (single source of truth: `Pool::winnings_for` )

`winnings = bet_weight × (total_pool – winning_side_stake) / total_weighted_winning` (u128 intermediate, integer floor). `gross = principal + winnings` , `fee = gross × fee_bps / 10000` , `net = gross – fee` . Time-weighting at deposit: `multiplier = max(10% floor, (lock – now)/window)` linear decay (early bettors weighted higher). Rounding dust is cleared by `sweep_vault_dust` ( $\leq 0.001$  USDC).

### Highest-risk areas to focus on

**Note (pre-audit hardening).** Findings 2, 4, 5, 6 and 8 below were hardened in the repo before this scope was frozen; each is annotated **Addressed** with the commit. We ask CertiK to **verify the fixes** rather than assume them. Findings 1, 3, 7 and 9 are

review/design items with no code change requested.

1. **Fund conservation** — prove  $\Sigma(\text{net payouts} + \text{fees} + \text{dust}) \leq \text{vault}$  for the weighted formula with many winners, hedged/multiple deposits, and a Draw winner; confirm rounding can never overpay (last claimer must not fail).
2. **force\_close\_pool** — previously closed the pool account without checking the vault was empty, so misuse could strand USDC in an orphan vault. **Addressed** ( 7e78c6c ): the instruction now takes the vault account and requires `vault.amount == 0` ( `VaultNotEmpty` ), reading it by `pool.vault` so it still works on legacy corrupted-bump vaults. Verify the guard can't be bypassed.
3. **Authority = oracle + payer** — `resolve` / `resolve_with_winner` set the winner with no on-chain oracle and no timestamp guard (the `end_time` check was removed). Whoever holds the key decides every outcome and payout recipient.
4. **Refund / double-settle interaction** — `claim` , `refund` , `refund_bettor` , `close_losing_bet` all key off the same `UserBet` and set `claimed` / close it. Confirm no bet can be settled twice. `refund_bettor` previously did **not** decrement `pool.total_*` / `weighted_*` , leaving inflated denominators after a partial refund. **Addressed** ( 7e78c6c ): it now rolls back the refunded stake's `total_*` / `weighted_*` (saturating). Verify the payout denominators stay correct across partial-refund-then-claim.
5. **Tournament completion gap** — no instruction set tournament `status = Completed` or `winner` , yet `claim_tournament_prize` required both, so the prize vault was unclaimable. **Addressed** ( b028b47 ): added `resolve_tournament(winner)` — authority-signed, Registering/Active → Completed, sets `winner` , guarded against re-resolve and against Cancelled, and proves the winner is a registered participant via their participant PDA. Verify only a real participant can win and that it can't be double-resolved.
6. **fee\_bps bound** — `fee_bps` is a caller-supplied argument on `claim` . **Addressed** ( 7e78c6c ): bounded on-chain to `MAX_FEE_BPS = 1000` (10%); the real schedule tops out at 500. Verify the ceiling holds.
7. **sweep\_vault\_dust bound** — the  $\leq 1000$   $\mu$ USDC (0.001) cap is enforced on-chain ( `DUST_THRESHOLD = 1000` , `amount <= DUST_THRESHOLD` ). Confirm the comparison is tight and can't be used to drain more than dust.
8. **Arithmetic** — deposits/claims use `checked_*` (good); tournament `claim/refund` previously used `.unwrap()` (panic on overflow). **Addressed** ( e9baa26 ): those now return `PoolError::Overflow` via `.ok_or(...)?` . Confirm no remaining panic paths in the money instructions.
9. **PDA/seeds & bumps** — caller-supplied 32-byte ids; verify uniqueness and that `sweep` / `close` re-derive the vault via stored bump (historical "corrupted bump" pools are why `force_close_pool` exists).

## 2. Trust / centralization model (design review to include in the report)

- **One hot authority key controls all settlement.** `AUTHORITY_SECRET_KEY` (a plaintext JSON keypair in an env var, loaded into a long-running Node process) is the program authority and signs every state-changing instruction except the user's own `deposit` : `resolve` , `claim` , `refund` , `close` , `force-close` , `dust-sweep` , and `tournament payout` . No multisig / HSM / threshold signing.
- **Outcomes are decided off-chain with no on-chain proof.** The backend computes the winning side in TypeScript from external feeds (crypto price buffer, TheSportsDB, Polymarket/UMA-CTF, LLM fallback) and pushes it via `resolve*` . The program trusts whatever side the authority asserts. Synthetic prices are already used deliberately for refund logic, proving the price args are fully authority-controlled.
- **The authority is also the fee wallet**, and `fee_bps` is a runtime argument.
- **Same authority key across localhost / Railway-dev / Railway-prod** — a leak in the weakest environment compromises production funds. (Recommendation: separate keys per environment + move prod settlement to a multisig / hardened signer.)
- **Gasless deposit relay** ( `POST /prepare-gasless-deposit` ): the authority is the tx fee-payer and pays SOL/rent for user deposits — a SOL-drain / grieving vector without rate-limiting.

On-chain double-spend protection is real (a re-resolved pool / re-claimed bet reverts), but resolution/payout idempotency across retries is enforced off-chain (DB optimistic locks). The auditor should confirm the on-chain "already-settled" checks are sufficient on their own.

## 3. NOT built yet — future audit targets (documentation-only today)

Per `docs/UP-UTILITY-SPEC.md` , `docs/TOKENOMICS-PRESALE.md` , `docs/LAUNCH-CHECKLIST.md` — none of this is code:

- **UP Token** — planned Solana SPL token (10B supply), mint authority to be revoked.
- **Vesting / locks** — Streamflow (team cliff+vest, presale rounds, liquidity).
- **Multisig** — Squads (treasury, mint authority, sale proceeds).
- **Presale sale / claim contract** — the launch checklist flags "any funds-holding sale/vesting contract" as the audit hard-gate; none written yet.
- **Staking** and **buyback & burn** — planned, not built.

The earn-by-using "UP Coins" emission is 100% off-chain (Postgres ledger: `coinsBalance` / `EmissionConfig` / `CoinSpend` ), converting to the on-chain token only at TGE. Nothing to audit on-chain until these contracts are written.

## 4. OUT of scope for a smart-contract audit

| Area                   | Why it's out of scope                                                                                                                                                                        |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HyperLiquid            | External exchange. UpDown deploys no HL contract; it signs orders against HL via an agent-wallet (agent key AES-encrypted, server-side signing). HL's contracts are HL's to audit.           |
| Pacifica               | External Solana perp DEX. Trading is off-chain signed REST; the only on-chain touch is a deposit into <b>Pacifica's own</b> program ( <code>PCFA5iYgmqK6...</code> ), which is not UpDown's. |
| Privy embedded wallets | User key custody is handled by Privy (third-party MPC/TEE). No UpDown contract governs it.                                                                                                   |
| LI.FI bridge           | Quote/status only against LI.FI's API; execution deferred. No UpDown bridge contract exists.                                                                                                 |
| Backend / web app      | <code>apps/api</code> (Express), <code>apps/web</code> / <code>apps/terminal</code> (Next.js), <code>packages/exchange-*</code> adapters — off-chain TypeScript, no bytecode.                |

## 5. Recommended SEPARATE web / infra pentest (not a contract audit)

- Agent-key custody at rest ( `EXCHANGE_KEY_ENCRYPTION_SECRET` , AES scheme) and the server-side order-signing path — a compromise here risks user funds on HL.
- Admin panel authenticated only by an `x-admin-key` header in `sessionStorage` (now role-gated: super / read-only / marketing).
- Privy SSO cross-subdomain cookie/session handling.
- Withdrawal / order-routing endpoints, rate-limiting, anti-farming / re-linking.
- Pacifica server keypair storage; CSP / security-headers rollout.

## 6. Deliverables to hand CertiK

1. Repo access to `programs/parimutuel_pools/` (Rust) + `Anchor.toml` + build.
2. The compiled **IDL** and the deployed **program id(s)** per cluster.
3. This scope doc + a functional spec: pool/bet/resolution/payout/refund/tournament flows and the authority model (§3).
4. The existing test suite ( `tests/money_math.rs` ) and any integration tests.